Dukaan Pro - System Workflows (Version 3)

Step by step: how the system works and why each table is connected. Easy English. Now includes Release 1.2: the Online Store.

What is new in this version

Release 1.2: Online Store. Four new workflows (19 to 22): turn on the store, publish products, product photos, and the public customer view. Small notes were added to Onboarding, Subscription, Products, Purchase, Sale and Stock Adjustment. All Release 1 and 1.1 workflows are unchanged, so your finished Release 1 keeps working.

The 22 workflows

#	Workflow	Release	Main tables involved
1	Shop Onboarding	Release 1	shops, users, defaults, suppliers, shop_store_settings, cash_movements
2	Login and Shop Access Check	Release 1	users, shops
3	Subscription: Suspend and Renew	Release 1	shops
4	Set Up Products (Category, Brand, Product)	Release 1	categories, brands, products, stock_levels
5	Add a Supplier	Release 1	suppliers
6	Purchase (Stock In)	Release 1	shops, purchases, purchase_items, inventory_units, stock_levels, inventory_movements, supplier_payments, cash_movements
7	Pay a Supplier Later	Release 1	purchases, supplier_payments, cash_movements
8	Stock Adjustment, Damaged and Lost Stock	Release 1	stock_levels, inventory_units, inventory_movements
9	Create a Sale (Cash, Part Payment or Udhar)	Release 1	shops, sales, sale_items, inventory_units, stock_levels, inventory_movements, payments, cash_movements
10	Customer Pays Udhar	Release 1	sales, payments, cash_movements
11	Customer Ledger (Udhar History)	Release 1	customers, sales, payments
12	Cancel with Reason (Sale, Purchase, Payment)	Release 1	sales, purchases, payments, supplier_payments, cash_movements, inventory_units, stock_levels
13	Add and Cancel an Expense	Release 1	expenses, expense_categories, cash_movements
14	Dashboard (Simple)	Release 1	all (read only)
15	Return and Refund	Release 1.1	returns, return_items, refunds, inventory_units, stock_levels, cash_movements
16	Owner Investment and Withdrawal	Release 1.1	cash_movements
17	Cash Count (Reconciliation)	Release 1.1	cash_movements, cash_reconciliations
18	Date Filters, Detailed Reports and Export	Release 1.1	all (read only)
19	Turn On the Online Store (Store Settings)	Release 1.2	shop_store_settings, shops

#	Workflow	Release	Main tables involved
20	Publish Products to the Online Store	Release 1.2	products, product_images, categories, brands
21	Product Photos	Release 1.2	product_images, products
22	Customer Browses the Store (Public)	Release 1.2	shop_store_settings, shops, products, brands, categories, product_images, stock_levels, inventory_units

Rules behind every workflow

1. Every action is one database transaction

When one event changes many tables, all changes are saved together or not at all. If one step fails, everything is rolled back.

BEGIN	Validate	Write all rows	COMMIT	(or ROLLBACK on error)

2. Every private query is filtered by shop_id

For the dashboard, shop_id comes from the logged-in session, never from the browser request. For the public store, the shop comes from the slug, and the public API can only read the allowlisted fields of published products.

3. History is added, never edited or deleted

Stock and cash history are append-only. A mistake is fixed with Cancel with reason. Master data is deactivated, not deleted. Store settings are switched off, not deleted.

4. Double-tap safe

Sales, purchases and payments carry a client_request_id. The same request twice returns the first saved record.

5. Public is an allowlist (new)

Public responses are built from a short list of named fields. Cost, supplier, IMEI, profit, notes and customer data are never selected, so they cannot leak by mistake.

Two security zones

Zone	Who	Access	What it can reach
Public store (khalid-mobile.dukaanpro.com)	Anyone, no login	Read only, by slug	Published, active products and the public profile of an enabled, active shop
Dashboard (app.dukaanpro.com)	Shop owner	Login, role check, shop_id from the session	All private data of his own shop only

Workflow 1: Shop Onboarding

Release: Release 1 **Who:** SaaS admin, then the shop owner

Goal: Create a new shop so the owner can start working. Onboarding is admin-assisted in the MVP.

Admin creates shop	Admin creates owner	Defaults + Walk-in seller	Owner sets password	Opening data

Step by step

#	Owner / user does	System does
1	Admin enters shop name, phone, address and subscription end date.	Creates one row in **shops** (status ACTIVE). Counters next_sale_no and next_purchase_no start at 1.
2	Admin enters the owner's name and phone.	Creates a **users** row (role OWNER, status ACTIVE, no password yet) with a one-time setup token (stored as a hash, with an expiry) and shows the setup link.
3	(automatic)	Copies default categories, brands and expense categories into this shop. Also creates the default supplier **Walk-in seller** (is_system = true) for used phones.
4	(automatic)	Creates the shop's **shop_store_settings** row: a unique slug made from the shop name (khalid-mobile, khalid-mobile-2), display name copied from the shop name, store switched OFF.
5	Owner opens the link and types his own password.	Saves only **password_hash**. Clears the setup token. The admin never sees the password.
6	Owner enters opening data (optional): opening cash, old Udhar of customers, old dues of suppliers, existing stock.	Opening cash goes to **cash_movements**. Old balances go to **customers / suppliers**. Existing stock is entered as an 'opening stock' purchase.
7	Owner logs in.	See Workflow 2.

Tables connected, and why

Table	What happens	Why it is connected
shops	Insert 1 row	The shop is the tenant. It also holds the two counters for automatic numbers.
users	Insert 1 owner row	shop_id links the owner to his shop, so the system knows which data he may see. The token supports first-time setup and admin-assisted recovery.
categories, brands, expense_categories	Insert copies	Copies (not shared rows) let every shop rename or deactivate its own lists safely.
shop_store_settings	Insert (store OFF)	Every shop gets its public profile row from day one, so the slug exists and is unique. Nothing is public until the owner turns the store on.
suppliers	Insert 'Walk-in seller'	Used phones bought from people use a normal purchase from this supplier. No special table is needed.
cash_movements	Insert OPENING_CASH (IN)	Cash is a ledger, so the starting amount is also a movement.
customers, suppliers	Set opening_balance	Brings old Udhar and old dues in without inventing fake sales.
purchases (is_opening_stock = true)	Insert opening stock	Existing stock enters inventory the normal way, with no supplier payable and no cash movement.

Rules that protect the data

- Phone number must be unique across the platform.
- One owner per shop in the MVP.
- Recovery: admin creates a new setup link. Nobody ever sends a password.

> **Example**
>
> Admin creates **Khalid Mobile Center**. Khalid gets a link, sets his password, enters opening cash Rs 500,000 and starts. The shop already has the 'Walk-in seller' supplier and counters S-0001 / P-0001 ready.

Workflow 2: Login and Shop Access Check

Release: Release 1 **Who:** Shop owner

Goal: Identify who the user is and make sure he can only reach his own shop's data.

Phone + password	Find user	Check user status	Check shop status	Create session with shop_id	Every request filtered by shop_id

Step by step

#	Owner / user does	System does
1	Owner types phone number and password.	Looks up **users** by phone.
2	(automatic)	Compares the password with password_hash. If wrong, shows a general error ('phone or password is wrong').
3	(automatic)	Checks users.status = ACTIVE.
4	(automatic)	Follows users.shop_id to **shops** and checks shops.status = ACTIVE (and subscription not expired).
5	(automatic)	Creates a session / token that contains user_id and shop_id.
6	Owner uses the app.	Every database query adds **WHERE shop_id = session.shop_id**. The shop_id is never taken from the request sent by the browser.

Tables connected, and why

Table	What happens	Why it is connected
users	Read	Holds the phone, password_hash and user status. Authentication ('who are you?').
shops	Read	Holds the shop status and subscription date. Authorization ('what may you open?') depends on it.
(all other tables)	Filtered by shop_id	Business Rule #1: a shop can only access its own data.

Rules that protect the data

- User status and shop status are two separate checks.
- A suspended shop sees a clear 'subscription expired' message, not 'wrong password'.
- Business data is never touched by login or suspension.

Example

Khalid logs in. The system finds his user, sees shop 'Khalid Mobile Center' is ACTIVE, and puts shop_id in his session. When he opens Sales, the query only returns rows with his shop_id. Another shop's sales are invisible to him.

Workflow 3: Subscription: Suspend and Renew

Release: Release 1 **Who:** SaaS admin (and an automatic check)
Goal: Block access when the subscription ends, but keep all business data. Restore access after renewal.

Subscription ends	Shop becomes SUSPENDED	Login blocked	Owner renews	Admin sets ACTIVE

Step by step

#	Owner / user does	System does
1	Admin (or a daily automatic check) sees shops.subscription_ends_at is in the past.	Sets shops.status = SUSPENDED.
2	Owner tries to log in or use the app.	Shop check fails. The owner sees the 'subscription expired' message. No data is deleted or changed.
3	Customers open the shop's online store.	The public API checks shops.status. A suspended shop's store answers 'store not available'. Nothing is deleted.
4	Owner pays the renewal.	Admin records it outside the app (MVP).
5	Admin extends the date and sets status ACTIVE.	Updates **shops**. Owner can log in again and everything is exactly as he left it.

Tables connected, and why

Table	What happens	Why it is connected
shops	Update status and subscription_ends_at	Access is controlled only here. Keeping it on the shop (not on every table) is why suspension is instant and safe.
users	No change	User status stays ACTIVE. The user exists, but the shop's subscription blocks access (two separate concepts).

Rules that protect the data

- Suspended does not mean deleted.
- Subscription status is separate from business data.
- A suspended shop's online store is hidden automatically. When the shop is ACTIVE again the store returns exactly as it was.

Example

Khalid's subscription ends on 30 June. On 1 July the shop is SUSPENDED. On 5 July he renews. Admin sets ACTIVE and a new end date. His products, Udhar and cash history are all still there.

Workflow 4: Set Up Products (Category, Brand, Product)

Release: Release 1 **Who:** Shop owner

Goal: Tell the system what the shop sells and how each item should be tracked.

Choose / add category	Choose / add brand	Add product	Choose tracking type	Set default prices

Step by step

#	Owner / user does	System does
1	Owner picks a category (Mobile Phones, Chargers...) or creates one.	Reads / inserts **categories** for this shop.
2	Owner picks a brand (Samsung, Oppo...) or creates one.	Reads / inserts **brands** for this shop.
3	Owner enters name (for example Samsung A15) and optional details: model, color, RAM, storage, SKU, barcode, description, image.	Prepares a **products** row.
4	Owner chooses tracking type: SERIALIZED (phones, IMEI) or QUANTITY (chargers, covers).	Saves tracking_type. This decides how stock is handled everywhere else.
5	Owner enters default purchase price, default selling price and low-stock level.	Saves the product. If it is QUANTITY, also creates a **stock_levels** row with quantity 0.
6	Later: owner no longer sells it.	Sets is_active = false. The product is never deleted.

Tables connected, and why

Table	What happens	Why it is connected
categories	Read / insert	A product belongs to one category. This allows reports by category and renaming a category without touching products.
brands	Read / insert	Same idea for brands. Brand is a separate table (not plain text) so spelling stays consistent.
products	Insert	The item 'type or model'. It is the master record that purchases, stock and sales point to.
stock_levels	Insert (QUANTITY products only)	Holds the current quantity of chargers and covers. Phones do not need it because each phone is its own row in inventory_units.

Rules that protect the data

- Product name should be unique per shop.
- tracking_type cannot be changed after the product has any transaction.
- Default prices are suggestions. Real prices are saved in purchases and sales.
- New products are NOT published (is_published = false). Publishing to the online store is a separate choice (Workflow 20).

Example

Khalid creates 'Samsung A15' (Mobile Phones, Samsung, SERIALIZED, default sell Rs 100,000) and 'Fast Charger' (Chargers, Anker, QUANTITY, default sell Rs 800). The charger also gets a stock_levels row with quantity 0.

Workflow 5: Add a Supplier

Release: Release 1 **Who:** Shop owner
Goal: Save the people or companies the shop buys from.

Enter name and phone	Optional address and notes	Optional opening balance	Save

Step by step

#	Owner / user does	System does
1	Owner enters supplier name, phone, address, notes.	Inserts a **suppliers** row for this shop.
2	Owner enters the old amount he already owes (if any).	Saves it in suppliers.opening_balance.
3	Used phones from a person.	Use the default supplier **Walk-in seller**. Do not create a new supplier for each person.
4	Later: supplier is no longer used.	Sets is_active = false. Old purchases keep this supplier.

Tables connected, and why

Table	What happens	Why it is connected
suppliers	Insert	Purchases and supplier payments point to a supplier so the system can show how much is owed to each one.

Rules that protect the data

- Suppliers are deactivated, never deleted. 'Walk-in seller' (is_system) cannot be deleted.
- The same supplier can supply many products and the same product can come from many suppliers (the link is purchase_items).

> **Example**
>
> Khalid adds 'ABC Mobiles, Peshawar'. He already owes them Rs 50,000 from before, so opening_balance = 50,000.

Workflow 6: Purchase (Stock In)

Release: Release 1 **Who:** Shop owner

Goal: Record goods bought from a supplier, add them to stock, and record how much was paid. This is the most important stock-in flow.

Pick supplier	Add items and cost	IMEIs (type or paste)	Enter paid now	Save (one transaction)	Stock + payable updated

Step by step

#	Owner / user does	System does
1	Owner picks the supplier. He can enter the supplier's own invoice number (optional) and the date.	Reads **suppliers**. The system number P-0001 is made automatically.
2	Owner adds items: product, quantity, unit cost.	Reads **products**. Backend calculates totals itself.
3	For phones, owner types or pastes one IMEI per phone and chooses NEW or USED. Count must equal quantity.	Checks every IMEI is not already in **inventory_units** for this shop.
4	Owner enters how much he pays now: all, part, or nothing.	If paid is more than current cash, shows a warning. Owner can confirm to continue.
5	Owner presses Save.	The screen sends a client_request_id. **BEGIN transaction**. If the same id was already saved, the old purchase is returned (no double save).
6	(automatic)	Reads and increases shops.next_purchase_no. Inserts **purchases** and one **purchase_items** row per line.
7	(automatic) each phone	Inserts **inventory_units** (IN_STOCK, unit_cost, condition) and **inventory_movements** (PURCHASE, +1).
8	(automatic) each accessory line	Updates **stock_levels** (quantity and average cost). Inserts **inventory_movements** (PURCHASE, +qty).
9	(automatic) if paid > 0	Inserts **supplier_payments** and **cash_movements** (SUPPLIER_PAYMENT, OUT; is_overdraft = true if the owner confirmed the warning). **COMMIT**.

Tables connected, and why

Table	What happens	Why it is connected
shops	Update counter	Gives the automatic purchase number, safely, inside the same transaction.
purchases	Insert	The invoice header: supplier, our number, supplier's number, total. Paid amount is not stored here.
purchase_items	Insert	Each product line with the real unit_cost, so old costs never change.
inventory_units	Insert (phones)	One row per physical phone. purchase_item_id lets you trace where it came from.
stock_levels	Update (accessories)	Fast current quantity and average cost.
inventory_movements	Insert	Why stock went up.
supplier_payments	Insert (if paid)	A purchase and its payment are separate. Payable = total - active payments.

Table	What happens	Why it is connected
cash_movements	Insert (if paid)	Only money actually paid reduces cash. Buying on credit does not.

Rules that protect the data

- At least one item. Quantity more than 0.
- IMEI must be unique (database rule).
- Average cost example: 40 chargers at Rs 500 + 20 at Rs 600 = about Rs 533.
- Cash warning is soft: confirm to continue. The cash row is marked is_overdraft.
- Wrong purchase? Use Cancel with reason (Workflow 12).
- If the shop has an online store, availability (In stock / Low / Out) follows the new stock automatically. It is calculated, not stored.

Example

Khalid buys 3 Samsung A15 (IMEIs 111, 222, 333, NEW) at Rs 90,000 and 40 chargers at Rs 500 from ABC Mobiles. He enters ABC's invoice 'AB-771'. System number: **P-0001**. Total = 290,000. He pays 100,000. Supplier owes **190,000**. Cash goes down 100,000.
A person sells him a USED phone for Rs 20,000: same screen, supplier 'Walk-in seller', condition USED, paid in full.

Workflow 7: Pay a Supplier Later

Release: Release 1 **Who:** Shop owner
Goal: Pay all or part of what is owed to a supplier.

Open supplier	See unpaid purchases	Choose purchase + amount	Save	Payable and cash reduce

Step by step

#	Owner / user does	System does
1	Owner opens a supplier.	Calculates payable: opening balance left + sum of (purchase total - active payments) for non-cancelled purchases.
2	Owner sees the unpaid purchases.	Reads **purchases** and **supplier_payments**.
3	Owner chooses a purchase (or the opening balance) and types the amount.	Checks amount is more than 0 and not more than what is left. If it is more than cash, shows a warning to confirm.
4	Owner presses Save.	**BEGIN**: inserts **supplier_payments** (purchase_id empty for opening balance), inserts **cash_movements** (SUPPLIER_PAYMENT, OUT). **COMMIT**.
5	Wrong amount?	Cancel with reason (Workflow 12): the payment is cancelled and a REVERSAL (IN) is added to cash.

Tables connected, and why

Table	What happens	Why it is connected
purchases	Read	Gives the total that must be paid for this invoice.
supplier_payments	Insert	Payments are separate rows so one purchase can have many payments over time.
cash_movements	Insert OUT	Real cash leaves the drawer, so the cash ledger records it.
suppliers	Read	Shows the supplier's total payable (sum over all purchases).

Rules that protect the data

- Payment cannot be more than what is left to pay.
- Records are never edited. Cancel with reason, then enter the correct payment.

Example

Khalid pays ABC Mobiles Rs 90,000 against the purchase with Rs 190,000 left. Now Rs 100,000 is left. Cash drops by 90,000.

Workflow 8: Stock Adjustment, Damaged and Lost Stock

Release: Release 1 **Who:** Shop owner

Goal: Fix stock when the physical count differs, or record damaged or lost items, always with a reason.

Pick product / IMEI	Choose type	Enter quantity and reason	Save	Stock and history updated

Step by step

#	Owner / user does	System does
1	Owner picks a product (and the IMEI for a phone).	Reads **products**, **stock_levels** or **inventory_units**.
2	Owner chooses ADJUSTMENT, DAMAGED or LOST.	Reason is required for all three.
3	Quantity items: owner types the real counted quantity.	System shows the difference (counted - system).
4	Owner presses Save.	**BEGIN**: inserts **inventory_movements** (type, quantity_change, reason); updates **stock_levels** or sets inventory_units.status to DAMAGED / LOST. **COMMIT**.

Tables connected, and why

Table	What happens	Why it is connected
stock_levels	Update (quantity items)	Current quantity must match the physical shelf. Cannot become negative.
inventory_units	Update status (phones)	A damaged or lost phone must not stay IN_STOCK, otherwise it could be sold.
inventory_movements	Insert	Stock never changes silently. The movement stores the reason, so you can answer 'why did stock change?' later.

Rules that protect the data

- Reason is required.
- Cannot reduce stock below zero.
- Only IN_STOCK phones can be marked damaged or lost.
- If the shop has an online store, availability (In stock / Low / Out) follows the new stock automatically. It is calculated, not stored.

Example

System says 37 chargers but the shelf has 35. Khalid saves ADJUSTMENT -2 with reason 'Counting difference'. stock_levels becomes 35 and a movement row is stored.

Workflow 9: Create a Sale (Cash, Part Payment or Udhar)

Release: Release 1 **Who:** Shop owner

Goal: Sell one or more products, record the payment, reduce stock, and record Udhar if the customer does not pay fully.

Customer (if needed)	Add items	Check stock / IMEI	Sale discount + price	Enter paid now	Save (one transaction)

Step by step

#	Owner / user does	System does
1	Owner picks a customer. Walk-in is allowed only when the sale has no phone.	Reads **customers**. A sale with a phone requires a customer (name + phone).
2	Owner adds items. For a phone he picks an IMEI that is IN_STOCK. For accessories he types a quantity.	Reads **products**, **inventory_units** (IN_STOCK) and **stock_levels** (enough?).
3	Price is suggested from the default selling price and can be changed. Owner can add one sale-level discount.	Backend calculates subtotal, discount and total. If a price is below cost, shows a warning (allowed).
4	Owner enters paid now (full, part, or zero) and an optional due date.	Outstanding = total - paid. If outstanding > 0 a customer is required.
5	Owner presses Save.	The screen sends a client_request_id. **BEGIN transaction**. A repeated id returns the saved sale (no double sale).
6	(automatic)	Reads and increases shops.next_sale_no. Inserts **sales** (S-0001...) and **sale_items** with unit_price and unit_cost.
7	(automatic) phones	Locks the unit, sets **inventory_units**.status = SOLD, inserts **inventory_movements** (SALE, -1).
8	(automatic) accessories	Reduces **stock_levels**, inserts **inventory_movements** (SALE, -qty).
9	(automatic) if paid > 0	Inserts **payments** and **cash_movements** (CUSTOMER_PAYMENT, IN). **COMMIT**. Any failure means ROLLBACK.
10	(after saving)	Owner can open or share a simple PDF receipt (shop name, S-number, items, IMEI, total, paid, balance).

Tables connected, and why

Table	What happens	Why it is connected
shops	Update counter	Automatic sale number inside the same transaction.
sales	Insert	The invoice header. Discount is here (sale-level only). Outstanding is calculated, not stored.
sale_items	Insert	What was bought. unit_price and unit_cost are saved at sale time, so profit never changes later. No item-level discount.
inventory_units	Update status	A phone is one physical item. IN_STOCK to SOLD stops it being sold twice.
stock_levels	Update	Reduces accessory quantity. A database check blocks negative stock.
inventory_movements	Insert	History of why stock went down.

Table	What happens	Why it is connected
payments	Insert (if paid)	A sale and its payment are separate. One sale can have many payments.
cash_movements	Insert (if paid)	Only money actually received increases cash. The unpaid part (Udhar) does not.
customers	Read	Needed for Udhar and purchase history.

Rules that protect the data

- At least one item. Cannot sell more than the stock. SOLD, DAMAGED, LOST or VOIDED IMEIs cannot be sold.
- Phone sale requires a customer. Udhar requires a customer.
- Selling below cost: warning only.
- Wrong sale? Cancel with reason (Workflow 12). Returns screen comes in Release 1.1.
- If the shop has an online store, availability (In stock / Low / Out) follows the new stock automatically. It is calculated, not stored.

Example

A) Walk-in accessories: 2 chargers at 800 + 1 cover at 500 = 2,100, discount 100, total **2,000**, paid 2,000. Cost 1,200, profit = 2,100 - 100 - 1,200 = **800**. Sale S-0001, no customer.

B) Phone on Udhar: Ali buys Samsung A15 (IMEI 111) for 100,000 (cost 90,000), pays 50,000. Sale S-0002. IMEI 111 becomes SOLD, cash IN 50,000, Ali's Udhar = **50,000**, profit = 10,000. If Khalid double-taps Save, only one sale is created.

Workflow 10: Customer Pays Udhar

Release: Release 1 **Who:** Shop owner
Goal: Record money received later from a customer who owes the shop.

Open customer	Type amount	System applies oldest first	Save	Udhar and cash updated

Step by step

#	Owner / user does	System does
1	Owner opens a customer profile.	Calculates current Udhar from non-cancelled sales and active payments.
2	Owner types the amount received.	Checks amount is more than 0 and not more than the customer's total Udhar.
3	(automatic)	Applies the amount to the oldest unpaid first: opening balance (oldest), then sales by date. Owner may choose a different sale instead.
4	Owner presses Save.	**BEGIN**: locks the sales rows, inserts one **payments** row per sale covered (all with the same receipt_group_id), inserts **cash_movements** (CUSTOMER_PAYMENT, IN). **COMMIT**.
5	Opening-balance part.	The payments row has sale_id empty (paying old Udhar).

Tables connected, and why

Table	What happens	Why it is connected
sales	Read (and lock)	Payments are linked to a specific sale. Locking stops two payments saved at once from exceeding the amount.
payments	Insert 1 or more	One row per sale covered. receipt_group_id ties them into one receipt. Old rows are never changed.
cash_movements	Insert IN	Real cash received. One movement per payment row.
customers	Read	Opening balance and total Udhar.

Rules that protect the data

- Payment cannot be more than total outstanding (no customer credit in the MVP).
- When outstanding is 0 the balance shows Rs 0 (settled).
- Wrong payment? Cancel with reason (Workflow 12).

Example

Ali owes 50,000 on S-0002 and 8,000 on S-0004 (older sale first). He pays 55,000: 50,000 goes to S-0002 and 5,000 to S-0004. Two payment rows, one receipt. Ali now owes **3,000**. If he tries to pay 60,000 (more than the 58,000 he owes), the system rejects it.

Workflow 11: Customer Ledger (Udhar History)

Release: Release 1 **Who:** Shop owner

Goal: Show how much a customer owes and, more important, why.

Open customer	System collects records	Sort by date	Show running balance

Step by step

#	Owner / user does	System does
1	Owner opens a customer's Ledger tab.	Reads all records for this customer in this shop.
2	(automatic)	Collects: opening balance, sales (add Udhar), payments (reduce). Cancelled sales and payments are shown as 'Cancelled' lines and do not count. Release 1.1 adds returns and refunds.
3	(automatic)	Sorts by date and calculates a running balance.
4	Owner reads the page.	Sees Current Udhar, purchase history, payment history, outstanding sales and due dates.

Tables connected, and why

Table	What happens	Why it is connected
customers	Read	Opening balance and profile.
sales	Read	Each unpaid or part-paid sale adds to what is owed. Holds due_date. Cancelled sales are shown but ignored in the balance.
payments	Read	Each active payment reduces the balance.
returns, refunds (R1.1)	Read	Added in Release 1.1: returns reduce the balance, refunds add back.

Rules that protect the data

- There is no separate ledger table. The ledger is built from real records, so it can never disagree with them.
- Balance tells how much he owes. The ledger tells why.

Example

Ali's ledger: Sale S001 +100,000 | Payment -50,000 | Payment -20,000 | Balance 30,000.

Workflow 12: Cancel with Reason (Sale, Purchase, Payment)

Release: Release 1 **Who:** Shop owner only
Goal: Fix a wrong entry without editing or deleting anything. The record is marked CANCELLED, stock and cash are put back with new rows, and the history stays.

Open the record	Press Cancel	Write reason	See the impact	Confirm	Reversal saved

Step by step

#	Owner / user does	System does
1	Owner opens a sale, purchase, customer payment or supplier payment and presses Cancel.	Only the OWNER role is allowed. Already-cancelled records cannot be cancelled again.
2	Owner writes a reason (required).	Reason is saved with the time.
3	(automatic)	Shows the impact before saving: stock that will come back or go out, cash that will be reversed, and the new Udhar or payable.
4	Owner confirms.	**BEGIN transaction**, then the steps below for the type chosen.
5	Cancel a SALE	**sales**.status = CANCELLED. Phones: **inventory_units** SOLD to IN_STOCK. Accessories: **stock_levels** + qty. **inventory_movements** (SALE_CANCEL). Each active **payments** row becomes CANCELLED and a **cash_movements** REVERSAL (OUT) is added.
6	Cancel a PURCHASE	Blocked if any phone from it is not IN_STOCK, or accessory stock would go negative. Otherwise **purchases**.status = CANCELLED, phones become VOIDED, **stock_levels** - qty, **inventory_movements** (PURCHASE_CANCEL), its **supplier_payments** are cancelled with cash REVERSAL (IN).
7	Cancel a PAYMENT	**payments** (or supplier_payments) status = CANCELLED and one **cash_movements** REVERSAL is added (OUT for a customer payment, IN for a supplier payment). The Udhar or payable comes back. **COMMIT**.

Tables connected, and why

Table	What happens	Why it is connected
sales / purchases / payments / supplier_payments	Update status, cancelled_at, cancel_reason	The original row stays, so reports and audits can see what happened and why.
cash_movements	Insert REVERSAL	The cash ledger is append-only. A reversal row (with reverses_movement_id) undoes the cash effect without erasing history.
inventory_units	Update status	Phones return to IN_STOCK (sale cancelled) or become VOIDED (purchase cancelled).
stock_levels	Update	Accessory quantity is put back or removed.
inventory_movements	Insert	SALE_CANCEL or PURCHASE_CANCEL keeps the stock history complete.

Rules that protect the data

- Only the owner can cancel. Reason is required.

- Cancelled records are ignored in Udhar, payable, profit and reports, but remain visible.
- If a reversal makes cash go below zero, show a warning and let the owner confirm (is_overdraft).
- After cancelling, enter the correct record as a new one.

Example

Khalid sold Ali a phone for 100,000 but typed the wrong IMEI. He cancels S-0002 with the reason 'Wrong IMEI'. IMEI 111 is IN_STOCK again, Ali's payment of 50,000 is cancelled and a REVERSAL OUT 50,000 is added to cash (Khalid gives the money back). Then he enters a new sale S-0005 with the correct IMEI.

Workflow 13: Add and Cancel an Expense

Release: Release 1 **Who:** Shop owner

Goal: Record the cost of running the shop (rent, electricity, salary...). Buying stock is not an expense.

Choose category	Amount + date + note	Save	Cash reduces	(Later) Cancel if wrong

Step by step

#	Owner / user does	System does
1	Owner picks an expense category (or creates one).	Reads **expense_categories**.
2	Owner enters amount, date and optional note.	If the amount is more than current cash, shows a warning to confirm.
3	Owner presses Save.	**BEGIN**: inserts **expenses** (ACTIVE), inserts **cash_movements** (EXPENSE, OUT). **COMMIT**.
4	Later: owner cancels a wrong expense and writes a reason.	Sets expenses.status = CANCELLED (with reason) and adds a **cash_movements** REVERSAL (IN).

Tables connected, and why

Table	What happens	Why it is connected
expense_categories	Read / insert	Groups expenses (Rent, Electricity...) for the expense report. Shop-specific.
expenses	Insert / update status	The operating cost. Used for net profit.
cash_movements	Insert OUT, and IN on cancel	Expenses are paid in cash, so cash must change. The ledger is append-only, so a cancel adds a reversal row.

Rules that protect the data

- Expenses are cancelled with a reason, never deleted.
- Net profit uses only ACTIVE expenses.
- Cash warning is soft (is_overdraft).

Example

Khalid saves Electricity Rs 5,000. Cash goes down 5,000 and net profit goes down 5,000. He typed it twice by mistake, so he cancels one: status CANCELLED and cash +5,000 reversal.

Workflow 14: Dashboard (Simple)

Release: Release 1 **Who:** Shop owner

Goal: Show the owner a quick picture of the shop right now. Everything is calculated from records and filtered by shop_id.

Open app	Query tables for this shop	Calculate	Show 7 numbers

Step by step

#	Owner / user does	System does
1	Owner opens the Dashboard (default: Today).	Every query starts with WHERE shop_id = my shop.
2	(automatic)	Reads the tables below. Cancelled sales, purchases, payments and expenses are not counted.
3	Owner taps a number.	Shows the matching list (for example today's sales, or customers who owe).

Where each dashboard number comes from

Dashboard item	Tables used	How it is calculated
Today's Sales	sales	Sum of non-cancelled sale totals today.
Today's Net Profit	sale_items, sales, expenses	Gross profit = revenue - discount - cost (unit_cost x quantity). Net profit = gross profit - ACTIVE expenses.
Customer Udhar	customers, sales, payments	Sum of opening balances left + outstanding of all non-cancelled sales.
Supplier Payable	suppliers, purchases, supplier_payments	Sum of opening balances left + (purchase totals - active supplier payments).
Today's Expenses	expenses	Sum of ACTIVE expenses today.
Low-stock Products	products, stock_levels, inventory_units	Accessories: quantity_on_hand <= threshold. Phones: count of IN_STOCK units <= threshold.
Cash Position	cash_movements	Sum of IN minus sum of OUT (reversals included).

Rules that protect the data

- Sales revenue, money received, Udhar, supplier payable, expenses and cash are different numbers and are never mixed.
- Date filters (week, month, custom) and detailed reports come in Release 1.1.

Example

At 6 PM: Today's sales Rs 120,000, net profit Rs 14,000, customer Udhar Rs 300,000, supplier payable Rs 450,000, expenses Rs 5,000, low-stock items 4, cash Rs 478,500.

Release 1.1 Workflows

Designed now, built after Release 1 has been used by a real shop.

Workflow 15: Return and Refund

Release: Release 1.1 **Who:** Shop owner

Goal: Take back an item, put it back in stock (or mark it damaged), and fix Udhar and cash. Until this release, owners use Cancel sale for mistakes.

Find original sale	Choose items to return	Reason + condition	Return value + refund	Save (one transaction)

Step by step

#	Owner / user does	System does
1	Owner finds the original sale (not cancelled).	Reads **sales** and **sale_items**.
2	Owner selects items and quantities (or the IMEI) to return.	Checks quantity is not more than sold minus already returned. An IMEI can be returned only once.
3	Owner chooses GOOD or DAMAGED and writes the reason (required).	Return value = unit_price x quantity minus that item's share of the sale discount.
4	System shows how the value is settled.	First it cancels outstanding Udhar of that sale. The rest was already paid, so up to that amount can be refunded.
5	Owner enters the refund amount (can be zero) and saves.	**BEGIN transaction**.
6	(automatic)	Inserts **returns** (return_amount) and **return_items**.
7	(automatic) GOOD items	Phone: IN_STOCK again. Accessories: stock_levels + qty. **inventory_movements** (CUSTOMER_RETURN).
8	(automatic) DAMAGED items	Phone: status DAMAGED. Accessories: not added to sellable stock. A DAMAGED movement is recorded.
9	(automatic) if refund > 0	Inserts **refunds** and **cash_movements** (REFUND, OUT). **COMMIT**.

Tables connected, and why

Table	What happens	Why it is connected
sales, sale_items	Read	The return must point to the original sale and item, so we know what was sold, at what price and cost.
returns	Insert	The return header. Linked to the sale; the sale itself is not touched.
return_items	Insert	Which items and how many. Unique on inventory_unit_id so one IMEI cannot be returned twice.
inventory_units / stock_levels	Update	Returned good items become sellable again. Damaged items do not.
inventory_movements	Insert	History of why stock changed.
refunds	Insert	Money given back is a new record, not a deletion of the old payment.
cash_movements	Insert OUT	A refund reduces real cash, and cash must not go below zero.

Rules that protect the data

- Original sale and payments are never edited or deleted.
- Return reason is required.
- Refund cannot be more than the money actually paid (minus earlier refunds).
- Outstanding after returns = sale total - returns - active payments + refunds.
- A sale that has returns cannot be cancelled.

Example

Ali returns phone IMEI 111 (GOOD). Sale was 100,000; he paid 70,000 and owes 30,000. Return value = 100,000. The 30,000 Udhar is cancelled. Khalid refunds 70,000 cash. Check: 100,000 - 100,000 - 70,000 + 70,000 = **0**. IMEI 111 is IN_STOCK again. Cash goes down by 70,000. Profit of this sale is reversed.

Workflow 16: Owner Investment and Withdrawal

Release: Release 1.1 **Who:** Shop owner

Goal: Record money the owner puts into the shop or takes out. These are not sales and not expenses.

Choose type	Amount + date + note	Save	Cash changes

Step by step

#	Owner / user does	System does
1	Owner chooses Investment or Withdrawal and types amount, date, note.	For a withdrawal, checks cash is enough.
2	Owner presses Save.	Inserts one **cash_movements** row: OWNER_INVESTMENT (IN) or OWNER_WITHDRAWAL (OUT).

Tables connected, and why

Table	What happens	Why it is connected
cash_movements	Insert	These events have no other record. The cash movement itself is the record. They are kept out of sales and expenses so profit stays correct.

Rules that protect the data

- Investment is not revenue. Withdrawal is not an operating expense.
- Cash cannot go below zero.

Example

Khalid adds Rs 200,000 of his own money (IN 200,000). Later he takes Rs 50,000 home (OUT 50,000). Profit does not change in either case.

Workflow 17: Cash Count (Reconciliation)

Release: Release 1.1 **Who:** Shop owner

Goal: Compare the system's cash with the real cash in the drawer, and record any difference.

System shows cash	Owner counts drawer	Compare	Reason if different	Save

Step by step

#	Owner / user does	System does
1	Owner opens Cash Count.	System shows system cash = sum of IN - sum of OUT from **cash_movements**.
2	Owner counts the real cash and types it.	System shows the difference.
3	If difference is not zero, owner must write a reason.	Reason is required.
4	Owner presses Save.	**BEGIN**: inserts **cash_reconciliations** (system, counted, difference, reason); inserts **cash_movements** (RECONCILIATION_ADJUSTMENT, IN or OUT) so system cash equals counted cash. **COMMIT**.

Tables connected, and why

Table	What happens	Why it is connected
cash_movements	Read, then insert adjustment	The system balance is calculated from it. The adjustment keeps the ledger honest instead of silently editing a number.
cash_reconciliations	Insert	A permanent record of every count, difference and reason, useful to find mistakes later.

Rules that protect the data

- Old cash rows are never edited.
- A reason is required whenever the difference is not zero.

Example

System cash 480,000. Counted 478,500. Difference -1,500, reason 'Wrong change given'. An OUT adjustment of 1,500 is added. System cash is now 478,500.

Workflow 18: Date Filters, Detailed Reports and Export

Release: Release 1.1 **Who:** Shop owner
Goal: Look at any period and see details. Export to PDF or CSV.

Choose report	Choose Today / Week / Month / Custom	Query tables	Show list and totals	Export

Step by step

#	Owner / user does	System does
1	Owner opens Reports and chooses a report and a date range.	Every query is filtered by shop_id and the date range.
2	(automatic)	Reads the tables below, ignoring cancelled records.
3	Owner reads totals and rows. He can export a PDF or CSV.	Export contains only this shop's data.

Tables connected, and why

Report	Tables used	What it shows
Sales report	sales, sale_items	Totals, discount, profit per sale.
Purchase report	purchases, purchase_items, supplier_payments	What was bought, paid and unpaid.
Expense report	expenses, expense_categories	Totals by category.
Profit report	sale_items, sales, expenses, returns	Gross and net profit for the period.
Customer report	customers, sales, payments	Who owes how much, due dates.
Supplier report	suppliers, purchases, supplier_payments	Payable per supplier.
Inventory report	products, stock_levels, inventory_units	Stock now, IMEIs in stock.
Best-selling products	sale_items	Sum of quantity per product in the period.

Rules that protect the data

- Reports never mix revenue, money received, Udhar and cash.
- If reports become slow later, add summary tables. In the MVP, direct queries with indexes are enough.

Example

Khalid picks 'This Month' in the Profit report: gross profit Rs 310,000, expenses Rs 120,000, net profit Rs 190,000. He exports a PDF for his records.

Release 1.2 Workflows: Online Store

One store website, one backend, one database. Every shop gets its own address (the slug). Shop owners keep managing products in the normal dashboard.

Workflow 19: Turn On the Online Store (Store Settings)

Release: Release 1.2 **Who:** Shop owner (admin for slug changes)
Goal: Give the shop a public profile and a store link, and switch the store on or off.

Open Store settings	Fill public profile	Choose stock display	Preview	Switch ON	Share the link

Step by step

#	Owner / user does	System does
1	Owner opens Store settings.	Reads **shop_store_settings**. The row already exists (made at onboarding, or by the backfill for old shops) with the store OFF.
2	Owner sees the store link, for example khalid-mobile.dukaanpro.com.	The slug is read-only for the owner. It was generated from the shop name; on a clash the system added -2, -3. Only an admin can change it, because shared links would break.
3	Owner fills the public profile: display name, about, logo, banner, WhatsApp number, phone, address, city, map link, opening hours, Facebook, Instagram.	Validates: WhatsApp = digits only with country code (923001234567); every link must start with http(s). The owner's login phone (users.phone) is never copied here.
4	Owner chooses how stock is shown: status only, or exact numbers.	Saves show_exact_stock. Default is status only.
5	Owner presses Preview.	Shows the store exactly as customers will see it, even while it is OFF. This uses a logged-in preview route, not the public one.
6	Owner switches the store ON.	Sets is_enabled = true. Warns if there is no WhatsApp number or no published product yet.
7	Owner copies the link or QR code and shares it on WhatsApp or Facebook.	Nothing is saved.
8	Later: owner switches the store OFF.	Sets is_enabled = false. The store disappears from the public API immediately. Data and slug stay.

Tables connected, and why

Table	What happens	Why it is connected
shop_store_settings	Read and update	The only place for public shop information and the on/off switch. Keeping it separate from *shops* makes it clear which fields customers can see.
shops	Read	The shop name is the starting value for the display name, and shops.status is checked every time the public API answers.
users	Not used	The owner's login phone is private. The store has its own phone and WhatsApp fields so it can never leak.
products	Read (count only)	Used only to warn the owner when no product is published.

Rules that protect the data

- Slug: lowercase letters, numbers and single hyphens, 3 to 40 characters, unique across all shops, no reserved words (www, app, api, admin ...). The database enforces this.
- Only an admin may change a slug.
- Links must start with http:// or https:// (blocks unsafe javascript: links).
- The store row is never deleted. Switch it off instead.

Example

Khalid opens Store settings. His link is **khalid-mobile.dukaanpro.com** (his friend's shop, also named Khalid Mobile, got khalid-mobile-2). He adds the WhatsApp number 923001234567, the address in Timergara and his opening hours, previews the store, and switches it ON. He pastes the link into his WhatsApp status.

Workflow 20: Publish Products to the Online Store

Release: Release 1.2 **Who:** Shop owner
Goal: Choose which products customers can see, and how each one is shown. Products are still created and priced in the normal dashboard.

Open product	Add photos	Public description + details	Price display	Publish	Check on the store

Step by step

#	Owner / user does	System does
1	Owner opens a product (or the product list with a Published column).	Reads **products** for this shop only.
2	Owner adds photos (see Workflow 21).	Reads **product_images** to show the current photos.
3	Owner writes a public description and checks the specs (model, color, RAM, storage). For phones he sets PTA status.	Saves public_description and pta_status. The private *description* is not touched and is never public.
4	Owner decides how the price is shown: normal price, or 'Contact for price'.	Saves contact_for_price. The selling price stays in the product. The public API hides it when contact_for_price is on.
5	Owner switches Publish ON (or selects many products and presses Publish).	Sets is_published = true. The database trigger sets published_at. Only active products can be published. Bulk publish is one transaction.
6	(automatic)	Nothing else changes: no stock, cost, purchase or sale record is touched. The product now appears in the public catalog if the store is ON.
7	Later: owner unpublishes it, or deactivates it.	is_published = false (published_at cleared), or is_active = false. It disappears from the store immediately.

Tables connected, and why

Table	What happens	Why it is connected
products	Update 4 fields	is_published, public_description, contact_for_price, pta_status. The product stays the single source of truth for name, brand, category and price.
product_images	Read	The cover photo is the first image. A product with no photo still works, but shows a placeholder.
categories, brands	Read	Their names are shown and used as filters on the store.
stock_levels, inventory_units	Not changed	Only read by the public API later, to calculate availability. Publishing never changes stock.

Rules that protect the data

- New products are never published by default.
- Inactive products never appear, even if is_published is true.
- Only the owner can publish or unpublish.
- Customers see only public_description, never the private description (it may contain internal notes).
- Purchase price, supplier, IMEI and profit are never part of a public response.
- A published product that sells out stays listed as 'Out of stock'.

Example

Khalid opens Samsung A15 (price Rs 45,000, 3 phones in stock). He uploads 2 photos, writes 'Brand new, box pack, 1 year warranty', sets PTA approved, and presses Publish. Two minutes later a customer sees it. He also publishes 'Fast Charger'. For a rare phone he turns on 'Contact for price'.

Workflow 21: Product Photos

Release: Release 1.2 **Who:** Shop owner
Goal: Add, reorder and remove the photos shown on the store.

Choose photo	App resizes it	Upload	Backend checks and stores	Save link	Reorder or delete

Step by step

#	Owner / user does	System does
1	Owner picks a photo from his phone.	The app shrinks it first (about 1600 px wide) so the upload is fast on mobile data.
2	Owner presses Upload.	Backend checks: owner is logged in, the product belongs to this shop, the file is a real JPEG, PNG or WebP (checked by content, not by name), size within the limit, and the product has fewer than 5 photos.
3	(automatic)	Strips hidden data such as GPS location from the photo, gives it a random file name, and saves it to storage under the shop and product folder. Gets a public https link.
4	(automatic)	Inserts **product_images** (url, sort_order = next number).
5	Owner drags photos to reorder, or marks one as cover.	Updates sort_order for that product's photos in one transaction. Lowest number = cover photo.
6	Owner deletes a photo.	Deletes the **product_images** row and removes the file from storage.

Tables connected, and why

Table	What happens	Why it is connected
product_images	Insert, update sort_order, delete	One row per photo. Photos are not financial records, so they may be deleted.
products	Read	Confirms the product exists and belongs to this shop before accepting the file.
shops	Read (via shop_id)	The shop id decides the storage folder and the ownership check.

Rules that protect the data

- Maximum 5 photos per product (service rule).
- Link must start with https:// (database rule).
- Never use the user's own file name. Use a random name.
- Remove GPS and other hidden data from every photo. Phone photos often contain the shop's exact location.
- Limit file size and accepted types.
- Pick the storage later (any S3-compatible bucket or an image service). The database only stores the link.

Example

Khalid photographs the A15 box in the shop and uploads 2 pictures. The first becomes the cover. Later he reorders them so the front view comes first, and deletes a blurry one. The product always keeps at least the photos he chose.

Workflow 22: Customer Browses the Store (Public)

Release: Release 1.2 **Who:** Customer (no login)
Goal: Let anyone see a shop's published products, prices and availability, and contact the shop, without seeing any private data.

Open store link	Find shop by slug	Check store is available	Browse and search	Product details	Message on WhatsApp

Step by step

#	Owner / user does	System does
1	Customer opens khalid-mobile.dukaanpro.com. No login.	The store website reads the address and takes the slug 'khalid-mobile'.
2	(automatic)	The store website asks the backend for GET /public/stores/khalid-mobile (from its server, not the customer's browser). The backend finds the row in **shop_store_settings** by slug (result cached about 60 seconds).
3	(automatic)	Checks: the store exists, is_enabled = true, and **shops.status** = ACTIVE. If any check fails, the answer is the same 'store not available' message, so nobody can tell the cases apart.
4	Customer sees the store page: name, about, logo, banner, WhatsApp, phone, address, map link, hours, social links.	Returns only the public profile fields from the allowlist.
5	Customer browses products, filters by category or brand, searches by name, goes to the next page.	GET /public/stores/:slug/products. Query: shop from the slug AND is_published AND is_active. Reads brand, category and cover photo. Availability comes from **stock_levels** (accessories) or one grouped count of IN_STOCK **inventory_units** (phones). Maximum 50 per page.
6	Customer opens a product.	GET /public/stores/:slug/products/:productId. The product must belong to this shop and be published and active, otherwise 404. A product id from another shop is also 404.
7	Customer taps 'Message on WhatsApp'.	Opens https://wa.me/923001234567 with a ready message containing the product name, price and link. No order is created and no customer data is saved.
8	(automatic)	Responses are cached for a short time. Requests are rate-limited per IP address.

Tables connected, and why

Table	What happens	Why it is connected
shop_store_settings	Read	Turns the slug into a shop, and supplies the public profile and the on/off switch.
shops	Read (status only)	A suspended shop's store is hidden.
products	Read (published and active only)	The catalog. Only the allowlisted fields are selected.
brands, categories	Read	Names and filters.
product_images	Read	Photos, ordered by sort_order.
stock_levels	Read	Availability for accessories.

Table	What happens	Why it is connected
inventory_units	Read (count of IN_STOCK)	Availability for phones. IMEI numbers are never selected.

Rules that protect the data

- The public API never reads: purchases, purchase_items, suppliers, supplier_payments, sales, sale_items, customers, payments, expenses, cash_movements, users.
- Every query uses an explicit field list (never 'select all, then remove').
- Never returned: purchase price, unit cost, average cost, profit, supplier, IMEI, SKU, barcode, private description, notes, shop_id, exact stock (unless the owner turned it on).
- Anyone may change the web address to another shop's slug. That is fine: they only see that shop's published products. The private dashboard still needs login and shop isolation.
- Automated tests must check that forbidden fields are absent from every public response and that a product id from another shop returns 404.

Example

Customer Hamza opens the link and searches 'A15'. He sees Samsung A15, 2 photos, Rs 45,000, PTA approved, **In stock**. Next week after sales only 2 remain: the badge says **Low stock**. After the last one is sold it says **Out of stock**. Khalid changed nothing. Hamza taps WhatsApp and talks to Khalid. For the 'Contact for price' phone he sees that text instead of a price.

Online Store API at a glance (Release 1.2)

Public routes (no login, read only, rate limited)

Route	Returns
GET /public/stores/:slug	Public store profile (allowlist). 404 'store not available' if missing, off, or shop suspended.
GET /public/stores/:slug/categories	Categories and brands that have at least one published product (for filters).
GET /public/stores/:slug/products?category=&brand=&q=&page=&limit=	Page of published, active products. Maximum limit 50. Newest first.
GET /public/stores/:slug/products/:productId	One product. 404 if it belongs to another shop, is unpublished or inactive.

Dashboard routes (login required, owner only, proposed)

Route	Purpose
GET and PATCH /store/settings	Read and update the public profile, show_exact_stock, and the on/off switch. Slug is read-only.
GET /store/preview/...	Same output as the public routes, for this shop, even while the store is OFF.
PATCH /products/:id	Existing route. Now also accepts is_published, public_description, contact_for_price, pta_status.
PATCH /products/publish	Bulk publish or unpublish: { ids: [...], isPublished: true }.
POST /products/:id/images	Upload one photo (max 5 per product).
PATCH /products/:id/images/order	Reorder photos. First = cover.
DELETE /products/:id/images/:imageId	Delete one photo.
PATCH /admin/shops/:shopId/store-slug	Admin only: change a slug.

Example product response (price is in paisa; the frontend divides by 100)

```
{ "id": "0192...", "name": "Samsung A15", "brand": "Samsung", "category": "Mobile Phones",
  "price": 4500000, "contactForPrice": false, "availability": "LOW_STOCK",
  "stockCount": null, "ptaStatus": "PTA_APPROVED", "condition": "NEW",
  "images": ["https://cdn.../a15-1.jpg", "https://cdn.../a15-2.jpg"],
  "description": "Brand new, box pack, 1 year warranty",
  "specs": { "ram": "6GB", "storage": "128GB", "color": "Blue" } }
```

NestJS structure and checks

- Separate module: public-store.module with its own controllers and services. Do not reuse ProductsService: it is built for logged-in owners and returns the internal product.
- Routes are marked @Public() so the global login guard skips them. A small StoreResolver (guard or interceptor) turns the slug into a shop id, and caches it for about 60 seconds.
- All queries use Prisma select with named fields, and responses are separate DTO classes.
- If you use a tenant filter with nestjs-cls, set the shop id from the slug so the filter also protects the public routes.
- Add @nestjs/throttler (per IP) and Cache-Control: public, s-maxage=60, stale-while-revalidate=300.
- Tests: (1) response never contains cost, supplier, IMEI, sku, barcode, private description or shop_id; (2) a product id from another shop returns 404; (3) store OFF and shop SUSPENDED return the same answer; (4) unpublished and inactive products never appear; (5) exact stock appears only when show_exact_stock is on.

Daily life: one day, many workflows

Time	What Khalid does	Workflow
10:00	Logs in	2
11:00	Buys phones and chargers from ABC Mobiles, pays part	6
12:00	Buys a used phone from a person (Walk-in seller)	6
1:00 PM	Sells chargers and covers to a walk-in customer	9
2:00 PM	Photographs the new A15 phones and publishes them on his store	20, 21
2:30 PM	A customer, Hamza, finds the A15 on the store and messages on WhatsApp	22
3:00 PM	Sells a phone to Ali on Udhar. The store badge changes from In stock to Low stock by itself	9, 22
4:00 PM	Ali pays part of his Udhar	10
5:00 PM	Pays the electricity bill	13
6:00 PM	Cancels a wrong sale with a reason	12
9:00 PM	Checks the dashboard	14
(Release 1.1)	Returns and refunds, cash count, reports and export	15 - 18

Quick Reference: which event writes which table

I = insert, **U** = update, **R** = read / check only, **+** = only when money is involved. Columns marked * are Release 1.1 tables. The public store only ever **reads** (last row) and only the allowlisted fields. It never touches purchases, suppliers, sales, customers, payments, expenses, cash or users.

Event	shops	store_settings	products	product_images	stock_levels	inv_units	inv_move-ments	purch-ases	purch_items	supp_pay-ments	sales	sale_items	pay-ments	expenses	cash_move-ments	returns*	return_items*	refunds*	cash_recon*
Onboarding	U	I			U	I	I	I	I						I				
Purchase	U		R		U	I	I	I	I	I+					I+				
Pay supplier								R		I					I				
Stock adjust / damaged / lost					U	U	I												
Sale	U		R		U	U	I				I	I	I+		I+				
Customer pays Udhar											R		I		I				
Cancel sale					U	U	I				U	R	U+		I+				
Cancel purchase					U	U	I	U	R	U+					I+				
Cancel payment										U			U		I				
Add expense														I	I				
Cancel expense														U	I				
R1.1 Return + refund					U	U	I				R	R			I+	I	I	I+	
R1.1 Owner invest / withdraw															I				
R1.1 Cash count															I				I
R1.2 Store settings / ON-OFF	R	U	R																
R1.2 Publish product			U																
R1.2 Upload / reorder photo			R	I															
R1.2 Customer browses (public)	R	R	R	R	R	R													